

Solution Code - Module 3-Core Java - JavapBytecodeInspection

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q37_JavapBytecodeInspection.java

```
1 | import java.io.BufferedReader;
2 | import java.io.File;
3 | import java.io.FileWriter;
4 | import java.io.InputStreamReader;
5 | import java.nio.file.Files;
6 | import java.nio.file.Path;
7 | import java.nio.file.Paths;
8 |
9 | public class Q37_JavapBytecodeInspection {
10 |     public static void main(String[] args) {
11 |         System.out.println("=== Using javap to Inspect Bytecode ===");
12 |
13 |         // Target source code representing the class to inspect
14 |         String sourceCode =
15 |             "public class SimpleMath {\n" +
16 |             "    public int addAndMultiply(int a, int b) {\n" +
17 |             "        int sum = a + b;\n" +
18 |             "        return sum * 2;\n" +
19 |             "    }\n" +
20 |             "}\n";
21 |
22 |         String targetClassName = "SimpleMath";
23 |         String srcFileName = targetClassName + ".java";
24 |         String classFileName = targetClassName + ".class";
25 |
26 |         try {
27 |             // 1. Write the source file
28 |             try (FileWriter writer = new FileWriter(srcFileName)) {
29 |                 writer.write(sourceCode);
30 |             }
31 |             System.out.println("1. Generated temporary class source file: " + srcFileName);
32 |
33 |             // Determine java compiler path or use system path
34 |             String sdkBinDir = findJavaBinDirectory();
35 |             String javacPath = sdkBinDir.isEmpty() ? "javac" : sdkBinDir + "javac";
36 |             String javapPath = sdkBinDir.isEmpty() ? "javap" : sdkBinDir + "javap";
37 |
38 |             // 2. Compile the class
39 |             System.out.println("2. Compiling " + srcFileName + " using " + javacPath + "...");
40 |             runCommand(javacPath, srcFileName);
41 |
42 |             if (!new File(classFileName).exists()) {
43 |                 throw new RuntimeException("Compilation failed: " + classFileName + " not generated.");
44 |             }
45 |         } catch (Exception e) {
46 |             System.out.println("Error: " + e.getMessage());
47 |         }
```

Solution Code - Module 3-Core Java - JavapBytecodeInspection

```
44 |         }
45 |         System.out.println("    Compilation succeeded.");
46 |
47 |         // 3. Run javap -c to inspect bytecode
48 |         System.out.println("3. Running javap -c " + targetClassName + "...");
49 |         |
System.out.println("-----");
50 |         runCommand(javapPath, "-c", targetClassName);
51 |         |
System.out.println("-----");
52 |
53 |         // 4. Interpretation notes
54 |         System.out.println("\n=== Interpretation of the Bytecode ===");
55 |         System.out.println("- iload_1 & iload_2: Load the int parameters 'a' and 'b' onto the operand
stack.");
56 |         System.out.println("- iadd: Pops two integers from the stack, adds them, and pushes the
result back.");
57 |         System.out.println("- istore_3: Pops the sum result and stores it in local variable 3
('sum').");
58 |         System.out.println("- iload_3: Loads the local variable 3 ('sum') onto the stack.");
59 |         System.out.println("- iconst_2: Pushes the integer constant '2' onto the stack.");
60 |         System.out.println("- imul: Pops two values (sum and 2) from the stack, multiplies them, and
pushes the result.");
61 |         System.out.println("- ireturn: Returns the integer result from the method.");
62 |
63 |     } catch (Exception e) {
64 |         System.err.println("Execution failed: " + e.getMessage());
65 |         e.printStackTrace();
66 |     } finally {
67 |         // Cleanup files
68 |         try {
69 |             Files.deleteIfExists(Paths.get(srcFileName));
70 |             Files.deleteIfExists(Paths.get(classFileName));
71 |             System.out.println("\n4. Cleaned up temporary files.");
72 |         } catch (Exception ignored) {}
73 |     }
74 | }
75 |
76 | private static String findJavaBinDirectory() {
77 |     // Try to check if we are in the installed Temurin path
78 |     String path = "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11-hotspot\\bin\\";
79 |     if (new File(path + "javac.exe").exists()) {
80 |         return path;
81 |     }
82 |     return "";
83 | }
84 |
85 | private static void runCommand(String... command) throws Exception {
86 |     ProcessBuilder pb = new ProcessBuilder(command);
87 |     pb.redirectErrorStream(true);
```

Solution Code - Module 3-Core Java - JavapBytecodeInspection

```
88 |         Process p = pb.start();
89 |         try (BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()))) {
90 |             String line;
91 |             while ((line = reader.readLine()) != null) {
92 |                 System.out.println("    " + line);
93 |             }
94 |         }
95 |         int exitCode = p.waitFor();
96 |         if (exitCode != 0) {
97 |             throw new RuntimeException("Command failed with exit code: " + exitCode);
98 |         }
99 |     }
100 | }
```